

УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ У НИШУ



COGMEN-JEPA

Предиктивно самонадгледано учење мултимодалних репрезентација за препознавање емоција у конверзацији

Мастер рад - студијско истраживачки рад

Студијски програм: Вештачка интелигенција и машинско учење

Студент:

Данило Милошевић,
број индекса 1732

Ментор:

Проф. др Јелена Николић

Ниш, октобар 2026. године

Садржај

1. Увод **4**

2. Аутоматско препознавање емоција **5**
2.1 Модели емоција 5
2.1.1 Дискретни модел емоција 5
2.1.2 Димензионални модел емоција 6
2.2 Методе аутоматског препознавања емоција 7
2.3 Скупови података и метрике 8
2.3.1 Скупови података 8
2.3.2 Метрике 9
2.4 Примене и изазови препознавања емоција 10

3. Основе неуронских мрежа за препознавање емоција **12**
3.1 Основе неуронских мрежа 12
3.1.1 Модел вештачког неурона 12
3.1.2 Вештачке неуронске мреже 16
3.1.3 Тренирање помоћу *backpropagation*-а 16
3.1.4 Иницијализација тежина неуронских мрежа 20
3.1.5 Регуларизација неуронских мрежа 21
3.2 Рекурентне неуронске мреже 22
3.2.1 Основна архитектура рекурентних неуронских мрежа 23
3.2.2 Трансформер архитектура 24
3.3 Граф неуронске мреже 26
3.3.1 Опште карактеристике граф неуронских мрежа 26
3.3.2 Варијације граф неуронских мрежа 28

4. Граф неуронске мреже за препознавање емоција **31**
4.1 Увод 31
4.2 Архитектура решења 32
4.2.1 Контекст екстрактор 32
4.2.2 Формирање графа 33
4.2.3 Релациона граф неуронска мрежа 34
4.2.4 Граф трансформер 34
4.2.5 Класификатор емоција 35
4.2.6 Хиперпараметри модела 35
4.3 Експериментални резултати 36
4.3.1 *IEMOCAP* 36
4.3.2 *MOSEI* 37
4.4 Оптимизација *COGMEN* модела 38
4.4.1 Бинаризација 38
4.4.2 *FP8* квантизација 39

5. JEPA **41**
5.1 Принцип рада и архитектура JEPA модела 41
5.2 Варијанте JEPA 41
5.3 Тренирање JEPA модела 41

6. Предлог рада **42**

7. Литература **43**
TODO – SREDI LITERATURU 43

1. Увод

Како је људски говор у великој мери обликован емоцијама, које утичу на тон гласа, паузе у говору и контекст разговора, као и на невербалне сигнале попут израза лица, од посебног је значаја да системи који комуницирају са људима буду у стању да аутоматски препознају људске емоције. Управо због комплексности људских емоција и начина њиховог изражавања, аутоматско препознавање емоција представља изузетно интересантну и изазовну област савремене вештачке интелигенције.

У наредним поглављима ћемо прво размотрити област аутоматског препознавања емоција, њен циљ, методе и технике које се у њој користе, као и различите примере примене. С обзиром на то да су се од свих техника најбоље показале неуронске мреже, у следећем поглављу ћемо обрадити принцип рада неуронских мрежа и начин на који се оне тренирају, а затим и сложеније архитектуре, као што су рекурентне мреже, трансформери и граф неуронске мреже. На крају тог поглавља ћемо објаснити и квантизацију, као једну од основних техника оптимизације перформанси неуронских мрежа. Циљ овог поглавља је да објасни основе неопходне за разумевање модела *COGMEN* – граф неуронске мреже која ће послужити као основни модел који ћемо касније модификовати. Након детаљне анализе модела *COGMEN*, обрадићемо архитектуру *JEPA* (*Joint Embedding Predictive Architecture*), коју ћемо користити у имплементацији како бисмо модел научили да предвиђа квалитетније латентне репрезентације мултимодалних *embedding*-а са улаза модела *COGMEN*. Помоћу ове архитектуре ћемо такође покушати да реконструишемо маскиране модале, као и будуће репрезентације на основу тренутних.

Овако модификован модел *COGMEN* ћемо затим тренирати над комбинованим скупом мултимодалних података за препознавање емоција, упоредити резултате са основном верзијом модела и анализирати квалитет латентних репрезентација, као и начин на који модел доноси одлуке.

Циљ овог рада је да се испита да ли је могуће применити архитектуру *JEPA* на мултимодалне податке, а затим и да се већ дефинисани модел за препознавање емоција тренира користећи нове латентне репрезентације. Осим покушаја да се модел унапреди тако да постигне боље укупне перформансе, циљ је и да се анализира да ли уз нове, квалитетније репрезентације модел може лакше да раздвоји сличне емоције, што му је у оригиналном раду представљало проблем. Поред тога, један од задатака је да се анализира како модел доноси закључке и зашто у појединим случајевима не успева да разликује сличне емоције. На крају ћемо, ослањајући се на отпорност архитектуре *JEPA* на маскирање делова репрезентација, покушати да реконструишемо маскиране репрезентације, како у домену модала, тако и у временском домену.

2. Аутоматско препознавање емоција

Аутоматско препознавање емоција је област вештачке интелигенције која се бави препознавањем емоционалног стања особе на основу изговорених речи, боје гласа, израза лица и говора тела, као и других физиолошких сигнала [1]. У оквиру ове области постоје различити циљеви, па се може препознавати тренутно емоционално стање једног или више говорника, али се може пратити и предвиђати и ток емоција током конверзације. У даљем раду ћемо се првенствено фокусирати на детекцију емоција у конверзацији.

У наредним одељцима ћемо анализирати моделе емоција и методе које се користе у оквиру аутоматског препознавања емоција, скупове података и метрике, као и реалне примене и изазове.

2.1 Модели емоција

Да бисмо уопште могли да предвиђамо људске емоције, прво је потребно дефинисати како ћемо описати емоције, односно одредити класе које је потребно предвидети. Теорију основних емоција, која је у великој мери обликовала ову област, развио је Пол Екман током 60-их и 70-их година прошлог века. Иако психолози покушавају да класификују емоције на различите начине, не постоји једногласно прихваћен модел емоција. Ипак, у области аутоматског препознавања емоција прихваћена су два модела – дискретни и димензионални модел емоција [2].

2.1.1 Дискретни модел емоција

Код овог модела, који се назива и категорички, емоције су подељене у фиксан број категорија. Два најчешћа дискретна модела емоција су Екманових шест основних емоција и Плутчиков точак емоција.

Екманов модел је широко прихваћен у области препознавања емоција. Овај модел дефинише шест основних емоција – бес, гађење, страх, срећу, тугу и изненађење [3] (слика 1). Ове емоције су издвојене на основу следећих принципа [2]:

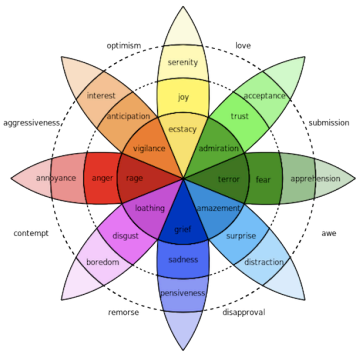
1. основне емоције морају бити засноване на људским инстинктима;
2. људи производе исте основне емоције у истим ситуацијама;
3. људи изражавају исте основне емоције са истим значењем;
4. основне емоције имају исти начин изражавања код свих људи.



Слика 1: Екманов модел емоција

Плутчиков модел се заснива на осам основних емоција – радости, поверењу, страху, изненађењу, тузи, очекивању, бесу и гађењу, као и на начину на који су ове емоције

међусобно повезане [4] (слика 2). Основне емоције су у точку распоређене у четири пара супротности, па су, на пример, радост и туга, као и страх и бес, супротне емоције. Поред тога, свака емоција се јавља у више нивоа интензитета, тако да јаче емоције заузимају центар точка, док се слабије налазе ка његовим крајевима, па се, на пример, очекивање може лако развити у будност (*vigilance*) [2]. Ове емоције се генерално могу сврстати у три поларитета (позитиван, негативан и неутралан), који се често користе у анализи сентимента [2].

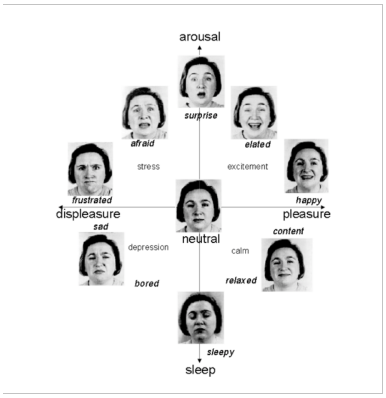


Слика 2: Плутчиков точак емоција

2.1.2 Димензионални модел емоција

Иако је дискретни модел интуитиван и погодан за означавање података, он има одређена ограничења. Већина дискретних модела не описује интензитет емоције, па се, на пример, благо незадовољство не разликује од изражене љутње. Такође је тешко представити помешане емоције, као и емоције које не припадају ниједној од унапред дефинисаних категорија. Због ових ограничења, као унапређење дискретног модела јавио се континуални, односно димензионални модел емоција. Овде ћемо првенствено анализирати *Valence-Arousal* модел, познат и као Раселов кружни модел, који емоције пресликава у континуални простор са две осе [5] (слика 3):

1. хоризонтална (x) оса, односно валенца, описује колико је емоција позитивна или негативна;
2. вертикална (y) оса, односно узбуђење (*arousal*), описује ниво активације, тј. интензитет емоције.



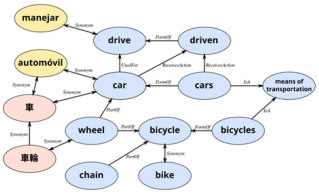
Слика 3: Раселов димензионални модел емоција

У оваквом простору свака емоција представља једну тачку. Срећа се, на пример, налази у делу простора са позитивном валенцом и високим узбуђењем, туга у делу са негативном валенцом и ниским узбуђењем, док бес има негативну валенцу и високо узбуђење. На овај начин је могуће представити и интензитет емоције, као и стања која се налазе између дискретних категорија. Раселов модел се често проширује и трећом димензијом, доминантношћу (*dominance*), која описује у којој мери особа има осећај контроле над ситуацијом, чиме се добија *PAD* (*Pleasure-Arousal-Dominance*) модел [6]. Мана димензионалног модела је то што је означавање података теже, а резултати се теже тумаче него код дискретних категорија [2].

2.2 Методе аутоматског препознавања емоција

Деценије научних истраживања у области аутоматске детекције емоција довеле су до развоја стотина различитих метода за препознавање емоција. Ове методе примењују различите технике из области обраде сигнала, рачунарског вида, обраде звука и машинског учења, међу којима се истичу технике засноване на Бајесовим мрежама, моделима мешавина Гаусових расподела, скривеним Марковљевим моделима и дубоким неуронским мрежама [1]. Методе се могу класификовати у три групе:

1. **Технике засноване на знању**
   Ове методе користе доменско знање, као и семантичку и синтаксичку анализу текста и изговора, како би детектовале одређене емоције. Неки од извора знања на које се овакве методе ослањају су *WordNet* [7], *SenticNet* [8], *ConceptNet* [9] и *EmotiNet* [10] (слика 4).



Слика 4: Семантичка мрежа ConceptNet

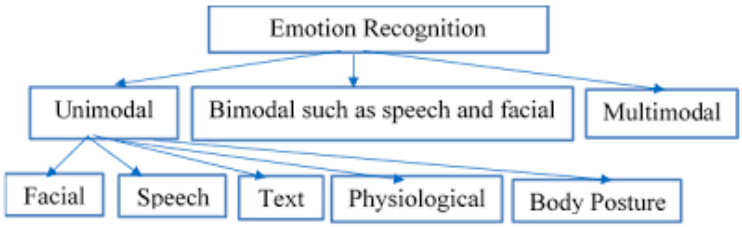
Ове технике се даље могу поделити у две категорије [11]:

- **технике засноване на речницима**, које у речнику проналазе кључне речи које носе емоције, а затим траже њихове синониме и антониме како би проширили почетни скуп емоционалних речи;
- **технике засноване на корпусима**, које крећу од почетног скупа емоционалних речи, а затим тај скуп проширују проналажењем других релевантних речи у великом корпусу текста.

2. **Статистичке методе**
   Ове методе користе различите алгоритме надгледаног машинског учења за аутоматско препознавање емоција. Најбоље резултате су постигле методе засноване на дубоком учењу, при чему се користе различите архитектуре, као што су конволуционе мреже, LSTM мреже или, као што ћемо видети у наредном поглављу, граф неуронске мреже [1].

3. **Хибридне методе**
   С обзиром на то да су технике засноване на знању довеле до открића принципа и хеуристичких правила која су била корисна за препознавање емоција, а да су статистичке методе знатно побољшале прецизност детекције емоција, јавила се идеја да се технике обе групе комбинују, чиме су настале хибридне методе [1].

Ове методе је могуће применити над једним или више типова података (слика 5). Прва истраживања у овој области ослањала су се искључиво на један извор информација, најчешће на људски говор или текст, односно користила су унимодални приступ. Овај приступ је постигао одређене резултате, али се убрзо показао ограниченим. Код текстуалног приступа проблем настаје услед чињенице да иста реченица може да носи другачије емоционално значење у зависности од начина на који је изговорена. Поред тога, у току разговора емоције једног говорника условљене су емоцијама и речима другог говорника, што унимодални приступ није био у стању да моделује. Код аудио записа говора резултати су бољи, али ни тада није увек могуће поуздано закључити емоције без израза лица говорника.



Слика 5: Подела метода препознавања емоција по броју модала

Услед ових ограничења развијени су мултимодални приступи препознавању емоција, који истовремено користе више извора информација (текст, аудио и видео податке) и на тај начин боље моделују емоционално стање говорника [1]. Модерни мултимодални системи комбинују технике обраде природног језика, рачунарског вида и обраде аудио сигнала и интегришу их у јединствену архитектуру која је способна да учи комплексне зависности из више извора података.

2.3 Скупови података и метрике

За тренирање и евалуацију модела за препознавање емоција потребни су скупови података у којима је сваки изговор означен одговарајућом емоцијом. Код мултимодалног препознавања емоција у конверзацији најчешће се користе скупови података *IEMOCAP*, *CMU-MOSEI* и *MELD* [12], чије су основне карактеристике приказане у табели 1.

2.3.1 Скупови података

IEMOCAP (*Interactive Emotional Dyadic Motion Capture*) [13] садржи око 12 сати снимака дијалога између парова глумаца, који су изводили унапред написане или импровизоване сцене. Сваки изговор је означен и категоријски и димензионално (валенца, активација и доминантност), а у литератури се најчешће користи подскуп са шест или четири емоције [12].

CMU-MOSEI (*CMU Multimodal Opinion Sentiment and Emotion Intensity*) [14] је један од највећих скупова података за анализу сентимента и препознавање емоција, састављен од видео снимака са *YouTube*-а у којима говорници износе мишљење о различитим

темама. За разлику од друга два скупа, снимци су монолози, а свака реченица је означена сентиментом и интензитетом шест Екманових емоција.

MELD (*Multimodal EmotionLines Dataset*) [15] садржи дијалоге из ТВ серије *Friends*, који најчешће укључују више од два говорника, што га чини тежим за моделовање. Изговори су означени са седам емоција и сентиментом.

Табела 1: Основне карактеристике скупова података за препознавање емоција

Скуп података	Извор	Говорника у разговору	Величина	Ознаке
IEMOCAP	глумци	2	151 дијалог, 7433 изговора	6 (4) емоција, валенца, активација, доминантност
CMU-MOSEI	*YouTube*	1 (монолог)	преко 23 000 реченица	сентимент, 6 емоција
MELD	серија *Friends*	2 и више	1433 дијалога, око 13 000 изговора	7 емоција, сентимент

2.3.2 Метрике

Најједноставнија метрика за оцену модела је тачност (*accuracy*), односно удео исправно класификованих изговора у укупном броју изговора. Међутим, у скуповима података за препознавање емоција класе нису подједнако заступљене, па се неутрално стање, на пример, јавља знатно чешће од гађења или страха. Модел који увек предвиђа најчешћу класу би тада могао да постигне релативно високу тачност, а да заправо не препознаје ниједну другу емоцију.

Због тога се користе мере које сваку класу посматрају посебно [16]. За класу c се дефинишу прецизност P_c, односно удео исправних предвиђања међу свим предвиђањима класе c, и одзив R_c, односно удео исправно препознатих изговора међу свим изговорима који заиста припадају класи c:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}$$

где су TP_c, FP_c и FN_c број тачно позитивних, лажно позитивних и лажно негативних предвиђања за класу c. F1 мера класе је хармонијска средина прецизности и одзива:

$$F1_c = \frac{2P_cR_c}{P_c + R_c}$$

Пондерисана F1 мера (*weighted F1*) добија се као просек F1 мера свих класа, где је свака класа пондерисана својим уделом у скупу података:

$$F1_w = \sum_{c=1}^{C} \frac{n_c}{N} F1_c$$

где је C број класа, n_c број изговора класе c, а N укупан број изговора. Тачност и пондерисана F1 мера су најчешће коришћене метрике у литератури за препознавање емоција у конверзацији [12], па ће се и у овом раду користити за поређење модела.

2.4 Примене и изазови препознавања емоција

Аутоматско препознавање емоција има примену у великом броју области, као што су здравство, аутомобилска индустрија, образовање, маркетинг и безбедност, као и многе друге (слика 6).

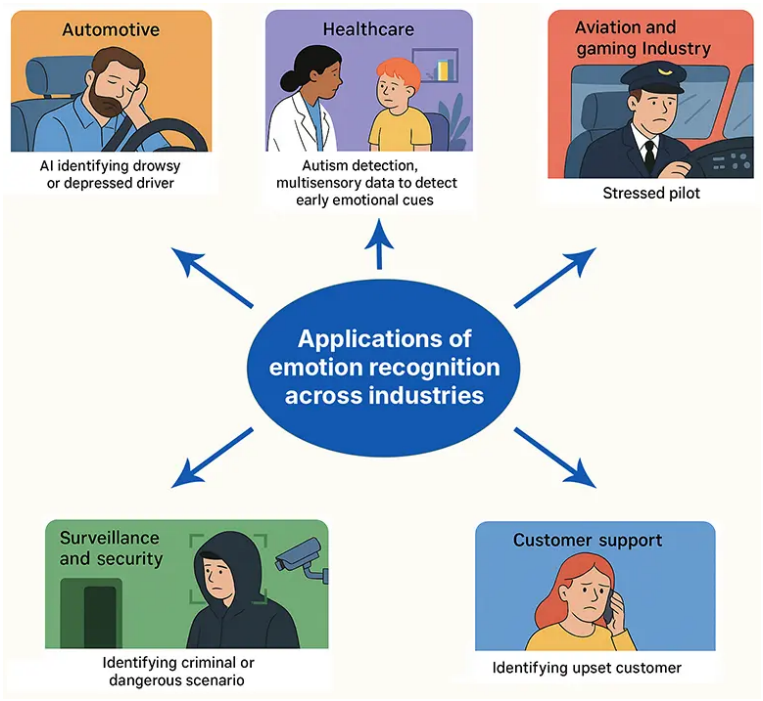
- **Здравство**
  У здравству се системи за препознавање емоција могу користити за аутоматско праћење менталног здравља пацијената, као и за детекцију раних знакова различитих психичких обољења, попут депресије или анксиозности. На пример, промене у говору, као што су спорији темпо и монотонији глас, показале су се као корисни показатељи депресије [17].

- **Аутомобилска индустрија**
  Системи за детекцију емоција примењују се на више начина у аутомобилској индустрији, а један од важнијих примера је детекција умора, поспаности или стреса код возача, како би систем могао да упозори возача или му предложи паузу и тако спречи саобраћајне несреће [18].

- **Образовање**
  Платформе за учење могу користити моделе за препознавање емоција како би утврдиле колико су ученици укључени у савладавање градива и да ли су збуњени или уморни, и у складу са тим прилагодиле начин предавања [19].

- **Маркетинг**
  Маркетиншке компаније могу користити аутоматско препознавање емоција како би утврдиле емоционалне реакције корисника на рекламе и на основу добијених информација кориговале кампање. На пример, анализом израза лица гледалаца током рекламе могуће је предвидети да ли ће им се реклама допасти, па чак и да ли ће купити рекламирани производ [20].

- **Безбедност**
  На местима са великим бројем људи, као што су тржни центри или аеродроми, могуће је користити аутоматско препознавање емоција како би се на време уочиле и спречиле потенцијалне опасности.



Слика 6: Примена аутоматског препознавања емоција

Поред бројних примена, аутоматско препознавање емоција се и даље суочава са значајним изазовима. Емоције су субјективне, па додељене класе емоција у скуповима података нису увек поуздане [13], док је сличне емоције, као што су срећа и узбуђење или бес и фрустрација, тешко разликовати чак и људима [12]. Поред тога, класе су у скуповима података неравномерно заступљене, а подаци снимљени у лабораторији не одражавају у потпуности спонтане разговоре из стварног живота. Препознавање емоција отвара и питања приватности и етике. У Европској унији је од 2025. године забрањена употреба система за препознавање емоција на радном месту и у образовним установама, осим из медицинских и безбедносних разлога [21].

3. Основе неуронских мрежа за препознавање емоција

Пре преласка на имплементацију конкретних модела за препознавање емоција објаснићемо појмове, технике и архитектуре неопходне за разумевање модела у наредном поглављу. Кренућемо од основних принципа рада неуронских мрежа и поступка њиховог тренирања. Како аудио записи говора и звука генерално нису фиксних димензија, односно модели који раде са говором, текстом или снимцима морају бити у стању да прихвате улазе различитог трајања, потребно је разумети **рекурентне неуронске мреже**, што ће нас на крају довести до трансформер архитектуре и *attention* механизма. Последња архитектура коју ћемо обрадити су граф неуронске мреже, као и њихове варијације – релационе граф неуронске мреже и граф *attention* мреже.

3.1 Основе неуронских мрежа

Дубоке неуронске мреже су се у протекле две деценије показале као најбоља техника машинског учења код сложених задатака, као што су рачунарски вид, разумевање дугих секвенци текста и обрада људског гласа, као и код многих других задатака код којих су стандардне методе захтевале велики број ручно дефинисаних правила, а притом постизале знатно лошије резултате [22].

Због тога ћемо прво објаснити основне принципе рада вештачких неурона, начин на који се они повезују у дубоке неуронске мреже, као и начин на који се овакве мреже затим тренирају за специфичне задатке.

3.1.1 Модел вештачког неурона

У биолошким системима неурони функционишу на принципу побуде, при чему се електрични импулси и хемијски сигнали преносе кроз нервни систем од једног неурона до другог. Уколико побуда неурона пређе одређени праг напона, неурон постаје активан и преноси сигнал даље.

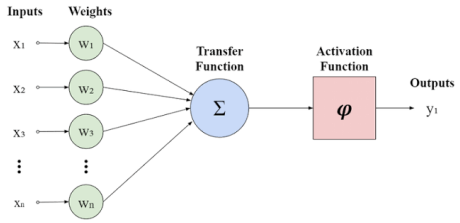
На основу оваквог модела биолошког неурона научници су током 40-их и 50-их година прошлог века моделовали вештачки неурон [23], [24]. Вештачки неурон на улазу прима низ улаза, односно сигнала

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

тј. улаз представља n-димензионални вектор колону. Излаз вештачког неурона добија се као линеарна комбинација вредности улаза, на коју се додаје и померај (слика 7):

$$y = f(x; w, b) = w^T x + b$$

Вектор w се назива вектор тежина, док је b *bias*, односно померај који се додаје линеарној комбинацији улаза. Уколико имамо слој од више неурона, тежине свих неурона чине матрицу тежина W, а помераји вектор b, па је излаз слоја $y = Wx + b$ [25]. Тежине и помераји се уче током тренирања.



Слика 7: Модел вештачког неурона

Претпоставимо да имамо два слоја неурона, тако да други слој на улазу прима излазе првог. У том случају излаз другог слоја добија се формулом

$$y = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

Можемо видети да је резултат поново линеарна функција улаза, тј. без обзира на број слојева неурони не могу моделовати комплексније функције.

Због овога се уводи **активациона функција**, чији је циљ да у систем неурона унесе нелинеарност. На овај начин неуронске мреже су у стању да уче и сложеније функције, па се неуронске мреже са бар једним скривеним слојем и нелинеарном активационом функцијом сматрају универзалним апроксиматорима функција [26].

Касније, током анализе тренирања мрежа, видећемо да се тренирање заснива на диференцирању, тако да активационе функције морају имати две особине:

1. **активациона функција мора бити нелинеарна**, тј. може имати линеарне делове, али не сме бити линеарна на целом домену, јер би се у супротном неуронска мрежа свела на линеарни модел;
2. **активациона функција мора бити диференцијабилна**, односно мора имати дефинисан извод (ово није строг захтев, што ћемо видети код функције *ReLU*, која није диференцијабилна у нули).

Неке од најпопуларнијих активационих функција су [25]:

1. **Сигмоидална функција**
   Дефинисана је једначином
   $$\sigma(x) = \frac{1}{1 + e^{-x}}$$
   и пресликава улаз у опсег $(0, 1)$ (слика 8a). Ова активациона функција била је међу првим коришћеним, с обзиром на то да има сличне карактеристике као активација неурона у биолошким системима. Касније се показала лошом код веома малих или веома великих улаза, где је извод функције близак нули, услед чега долази до нестајања градијената.

2. **Тангенс хиперболички**
   Дефинисан је једначином
   $$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$
   По облику је сличан сигмоидалној функцији, с обзиром на то да се може изразити преко ње, али за разлику од ње пресликава улаз у опсег $(-1, 1)$ (слика 8b). Као и код сигмоидалне, јавља се проблем нестајања градијената.

3. ***ReLU***
   Врло једноставна активациона функција, која се брзо рачуна, што је њена велика предност [27]. Дата је једначином

$$\text{ReLU}(x) = \max(0, x)$$

   Иако није диференцијабилна у 0, у пракси се показала изузетно добром (слика 8c). За позитивне улазе извод ове функције је увек 1, тако да се градијент не смањује током *backpropagation*-а, као што је случај код сигмоидалне функције и тангенса хиперболичког. Због тога *ReLU* ублажава проблем нестајања градијената и омогућава тренирање дубљих неуронских мрежа [25].

4. ***LeakyReLU***
   Код *ReLU*-а се за негативне вредности улаза на излазу увек добија 0, чиме губимо вредност као и градијент, односно неурон престаје да учи. Овај проблем се назива „умирање“ неурона. Решење овог проблема доноси *LeakyReLU* [28], која је дата формулом

$$\text{LeakyReLU}(x) = \max(0, x) + \alpha \min(0, x)$$

   односно позитивне вредности остају непромењене, док се негативне множе изабраном малом константом α (слика 8d).

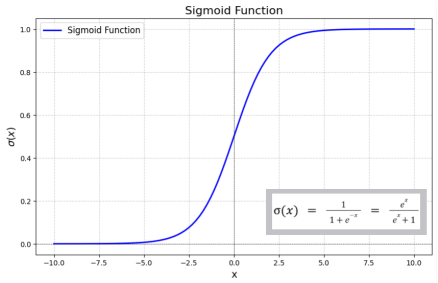
5. ***ELU***
   Претходне две функције имају недостатак где за вредност $x = 0$ извод функције није дефинисан. Овај проблем решава функција *ELU* [29], где позитивне вредности остају исте, док се негативне пресликавају експоненцијалном функцијом како би се добио гладак прелаз (слика 8e). За $\alpha = 1$ функција је диференцијабилна и у нули:

$$\text{ELU}(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

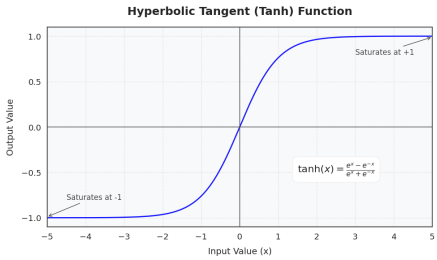
6. ***GELU***
   GELU [30] је глатка нелинеарна функција која улаз множи кумулативном функцијом расподеле нормалне расподеле $\Phi(x)$, односно $\text{GELU}(x) = x \cdot \Phi(x)$. Како је рачунање ове функције доста рачунски захтевно, користи се апроксимација

$$\text{GELU}(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715 \cdot x^3\right)\right)\right)$$

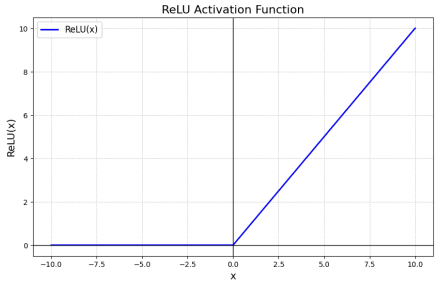
   Иако рачунски захтевна, ова функција је глатка и избегава проблем умирања неурона, јер градијент за негативне улазе није 0 (слика 8f). Нашла је примену у модерном дубоком учењу, поготово код трансформера.



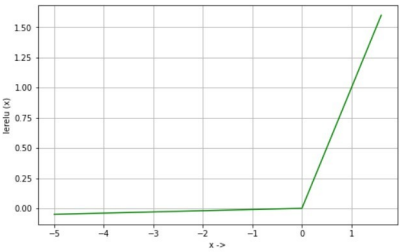
(a) Сигмоидална



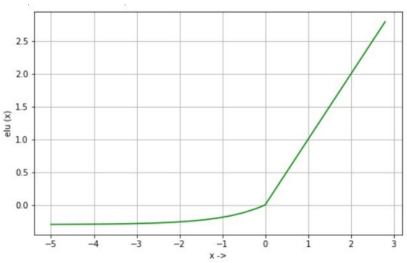
(b) Тангенс хиперболички



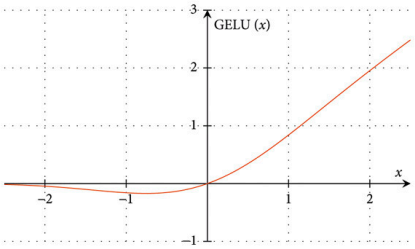
(c) *ReLU*



(d) *LeakyReLU*



(e) *ELU*



(f) *GELU*

Слика 8: Активационе функције

3.1.2 Вештачке неуронске мреже

Када смо дефинисали вештачки неурон, можемо конструисати вештачке неуронске мреже, које представљају скуп неурона подељених у слојеве који су међусобно повезани [25].

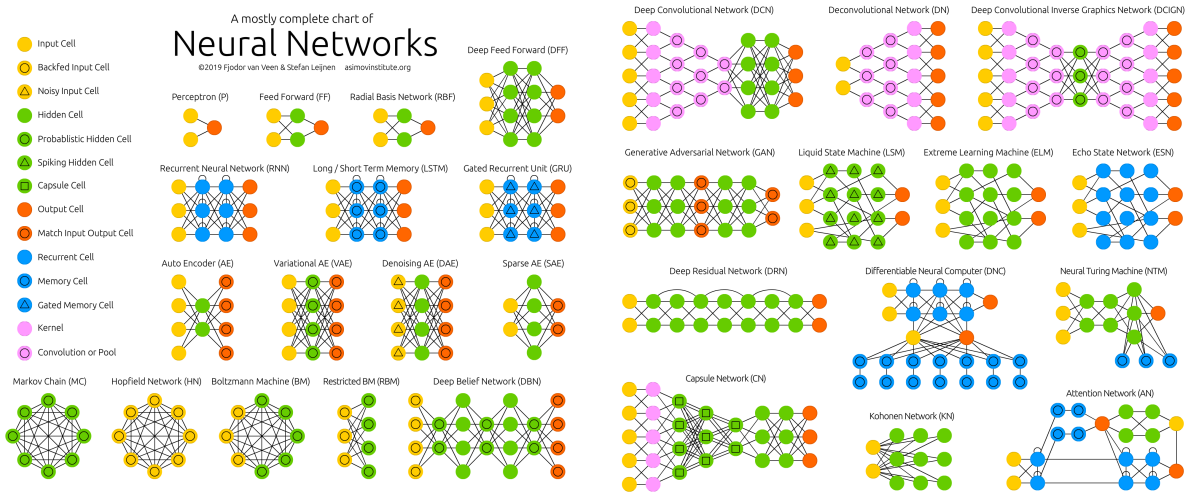
Први слој представља улазни слој, при чему у овом слоју имамо онолико неурона колико имамо улазних вредности. За разлику од осталих слојева, у улазном слоју неурони само прослеђују вредност следећем слоју, без икаквих модификација.

Након улазног слоја следи одређени број скривених слојева. Управо ови слојеви омогућавају неуронским мрежама да апроксимирају комплексне нелинеарне функције. Број неурона у сваком скривеном слоју, принцип рада неурона, повезаност између слојева, као и сам број слојева су произвољни и зависе од архитектуре неуронске мреже. Посебне врсте слојева, као што су конволуциони слојеви или *attention* слојеви код трансформера, специјалне везе као што су резидуалне везе, као и многе друге напредне технике неуронских мрежа примењују се управо у скривеним слојевима.

На крају имамо излазни слој, који прима излаз скривених слојева и, у зависности од типа задатка, одређује излаз мреже. У случају класификације, на крају се налази *softmax* активациона функција, која на основу излаза претходног слоја z генерише вектор вероватноћа чија је димензија једнака броју класа C:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Неки од типова неуронских мрежа, који се разликују управо по броју, врсти и повезаности слојева, приказани су на слици 9.



Слика 9: Типови неуронских мрежа [31]

3.1.3 Тренирање помоћу *backpropagation*-а

Након што дефинишемо архитектуру неуронске мреже, потребно је иницијализовати тежине мреже. За сада ћемо их поставити на насумичне вредности са униформном расподелом, а напредније технике ћемо видети у одељку 3.1.4. Тежине не смемо поставити на 0, нити на исту вредност, јер би тада сви неурони у слоју рачунали исти излаз и

добијали исти градијент, тако да би током тренирања остали исти, односно мрежа не би могла да научи различите карактеристике [25].

Кроз овакву мрежу пропуштамо улаз и на излазу добијамо одређене вредности. Како утврдити да ли је и колико неуронска мрежа погрешила?

Да бисмо на ово одговорили, уводи се **функција губитка** (*loss function*). Оваква функција узима стварну вредност излаза y и предвиђену вредност $\hat{y}$, а затим рачуна њихову „удаљеност“, у зависности од задатка. Круцијално је да ова функција буде диференцијабилна, како бисмо могли да одредимо како се губитак мења са променом тежина.

Погледаћемо неколико функција губитка [25]:

1. ***Cross-entropy* губитак**
   Користи се код класификационих задатака. Уколико имамо стварну вредност излаза y, дату као вектор у коме је вредност 1 само на позицији тачне класе, и предвиђене вероватноће $\hat{y}$ за C класа, овај губитак је дефинисан као

$$CE = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

   Како је $y_i = 0$ за све класе осим тачне класе c, од суме остаје само члан $-\log(\hat{y}_c)$. Можемо закључити да је губитак мањи уколико модел за тачну класу предвиђа велику вероватноћу, јер

$$\lim_{\hat{y}_c \to 1} -\log(\hat{y}_c) = 0$$

   док губитак тежи бесконачности када вероватноћа тачне класе тежи нули.

2. **Средња квадратна грешка**
   Користи се код регресионих задатака. Уколико имамо стварне вредности y_i и предвиђене вредности $\hat{y}_i$ за n примера, овај губитак је дефинисан као

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

   Због квадрирања, велике грешке имају знатно већи утицај на губитак од малих, па се модел посебно кажњава за велика одступања.

Када смо установили и функције губитка, можемо објаснити како се тренирају неуронске мреже [32]. Излаз слоја i означићемо са $a_i = \sigma(z_i)$, где је $z_i = W_i a_{i-1} + b_i$. Након пропуштања сигнала кроз целу мрежу, на крају добијамо излаз a_n и на основу излаза колики је губитак мреже. Да бисмо одредили како се мења губитак са променом тежина последњег слоја, рачунаћемо

$$\nabla W_n = \frac{\partial L}{\partial W_n} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial W_n}$$

при чему је L функција губитка, а W_n матрица тежина последњег слоја. Како одредити утицај претходних слојева на губитак? Тежине претходног слоја утичу на губитак само преко излаза тог слоја, па применом правила ланца добијамо

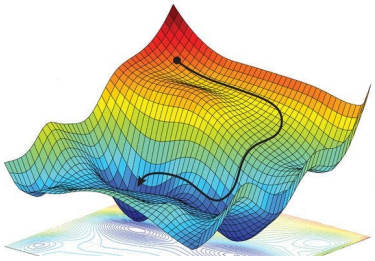
$$\nabla W_{n-1} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdot \frac{\partial a_{n-1}}{\partial W_{n-1}}$$

Уколико наставимо до првог слоја

$$\nabla W_1 = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdot \ldots \cdot \frac{\partial a_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial W_1}$$

Сваки од чланова $\frac{\partial a_{i+1}}{\partial a_i}$ зависи од тежина W_{i+1} и извода активационе функције, тако да код сигмоидалне функције и тангенса хиперболичког множењем великог броја малих извода долази до нестајања градијената. Градијенти се рачунају од последњег ка првом слоју, по чему је *backpropagation* и добио назив. На сличан начин се рачуна и градијент помераја (*bias*-а).

На основу добијених градијената вршимо ажурирање тежина слојева. Тренирање неуронских мрежа заправо представља проблем математичке оптимизације, где је циљ минимизација функције губитка (слика 10).



Слика 10: Минимизација функције губитка

Постоји више начина да ажурирамо тежине [33]:

1. **Стохастички градијентни спуст**
   Директно примењујемо градијенте

   $$W_i \leftarrow W_i - \eta \nabla W_i$$

   при чему је η стопа учења, којом скалирамо градијент како не бисмо правили превелике промене тежина. Градијент се не рачуна над целим скупом података, већ над насумично изабраним подскупом (*mini-batch*), по чему је метода и добила назив стохастички [34].

2. **Моментни спуст**
   Уводи се момент како бисмо оптимизовали градијентни спуст, односно убрзали конвергенцију тежина ка минимуму [35]. Идеја овог алгоритма је акумулирати градијенте током спуста и на тај начин акумулирати момент. Ово помаже на два начина – брже се конвергира, али се поред тога смањују осцилације и лакше се пролази кроз плитке локалне минимуме.

   $$v_t = \beta v_{t-1} + (1-\beta)\nabla W_i, \qquad W_i \leftarrow W_i - \eta v_t$$

   При чему β одређује колико утичу претходни градијенти, најчешће 0.9.

3. **Нестеров градијентни спуст**
   Представља модификацију моментног спуста, при чему се градијент не рачуна у тренутној тачки, већ у тачки у којој би се тежине нашле након примене момента [36].

   $$v_{t+1} = \beta v_t + \nabla L(W_i - \eta\beta v_t), \qquad W_i \leftarrow W_i - \eta v_{t+1}$$

На овај начин Нестеров спуст „гледа унапред“ градијент у следећем кораку и коригује путању како не би прескочио минимум. Код конвексних функција овакав спуст постиже теоријски оптималну стопу конвергенције.

4. ***Adam* оптимизатор**
   Уместо једног момента, ова метода користи два момента [37]:

   - први момент
   $$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla W_i$$
   - други момент
   $$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla W_i)^2$$

   Како моменти почињу од нуле, након овога се коригују

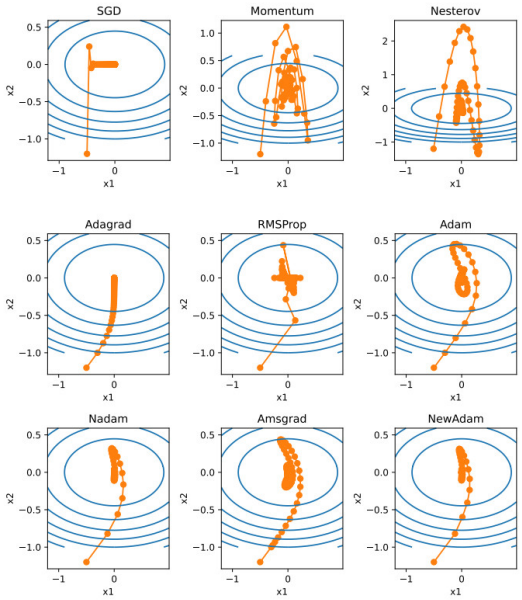
   $$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

   а затим се ажурирају тежине

   $$W_i \leftarrow W_i - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

   где је ϵ мала константа како не бисмо делили нулом. На овај начин сваки параметар има сопствену стопу учења. Представља један од најбржих и најчешће коришћених оптимизатора.

Путање којима различити оптимизатори долазе до минимума функције губитка приказане су на слици 11.



Слика 11: Путање различитих оптимизатора

Избор стопе учења значајно утиче на тренирање неуронских мрежа. Код премале стопе учења оптимизатор прави мале кораке, чиме се тежине прецизније подешавају,

али је тренирање доста спорије и може се заглавити у локалном минимуму. Превелика стопа учења убрзава тренирање, али оптимизатор може прескочити минимум, па губитак почиње да осцилује или чак да расте [25].

Због тога се уводи *scheduler* стопе учења. Идеја је да на почетку тренирања бирамо вишу стопу учења, како бисмо брже дошли до добрих резултата и избегли локалне минимуме, а да се затим са епохама тренирања стопа учења смањује. Тада оптимизатор прави мање кораке, прецизно подешавајући тежине модела. Стопа учења може опадати линеарно, у корацима или експоненцијално, као и по многим другим принципима [33].

3.1.4 Иницијализација тежина неуронских мрежа

Прве неуронске мреже су имале мали број слојева, па је тренирање оваквих модела било релативно једноставно. Међутим, са повећањем броја слојева показало се да је тренирање дубоких мрежа знатно теже, а један од узрока је и начин иницијализације тежина [38]. Разлог за отежано тренирање дубоких неуронских мрежа може се открити анализом излаза неурона са n_{in} улаза (без помераја)

$$z = \sum_{i=1}^{n_{in}} w_i x_i$$

Уколико претпоставимо да су тежине и улази међусобно независни, са очекиваном вредношћу 0, варијанса излаза је

$$\text{Var}(z) = n_{in} \cdot \text{Var}(w) \cdot \text{Var}(x)$$

Одавде видимо да уколико је $n_{in} \cdot \text{Var}(w) < 1$, варијанса излаза ће бити мања од варијансе улаза, а уколико је већа од 1, варијанса излаза ће бити већа. Како се ово понавља у сваком слоју, са повећањем дубине варијанса експоненцијално опада или расте, што онемогућава тренирање дубљих мрежа. Да би варијанса остала иста кроз слојеве, односно да би важило $\text{Var}(z) = \text{Var}(x)$, потребно је да буде

$$\text{Var}(w) = \frac{1}{n_{in}}$$

Слично разматрање за градијенте при *backpropagation*-у даје услов $\text{Var}(w) = \frac{1}{n_{out}}$, где је n_{out} број излаза слоја. Због ових услова уводе се две технике иницијализације тежина мреже:

1. ***Xavier* (*Glorot*) иницијализација**
   Како оба услова не могу бити испуњена истовремено, узима се компромис

   $$\text{Var}(w) = \frac{2}{n_{in} + n_{out}}$$

   Тежине се најчешће узимају из униформне расподеле

   $$w \sim U\left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right]$$

   чија је варијанса управо $\frac{2}{n_{in}+n_{out}}$. Ова иницијализација је изведена под претпоставком да је активациона функција приближно линеарна око нуле, тако да је погодна

за сигмоидалну функцију и тангенс хиперболички [38].

2. ***He* (*Kaiming*) иницијализација**
   Код *ReLU* функције, за негативне улазе излаз је 0, тако да у просеку половина неурона не преноси сигнал. Због тога се варијанса излаза преполови, па је потребно варијансу тежина удвостручити [39]:

$$\mathrm{Var}(w) = \frac{2}{n_{in}}, \qquad w \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

   Ова иницијализација се данас користи код већине мрежа са *ReLU* активационом функцијом и њеним варијацијама.

3.1.5 Регуларизација неуронских мрежа

Неуронске мреже имају велики број параметара, тако да су у стању да „запамте“ тренинг скуп уместо да науче опште обрасце у скупу података. Овај проблем се назива *overfitting* и лако се може уочити током тренирања, у случајевима када губитак над тренинг скупом наставља да опада, док губитак над валидационим скупом почиње да расте. Мерењем перформанси над тренинг скупом и тест скупом уочавамо да модел има добре перформансе над тренинг скупом, док су перформансе над тест скупом знатно лошије. Технике којима се ово спречава називају се технике регуларизације [25].

1. ***L2* регуларизација**
   У оквиру ове методе додајемо губитку члан којим кажњавамо велике тежине

$$L' = L + \frac{\lambda}{2}\|W\|_2^2$$

   при чему λ одређује јачину регуларизације. Како се у сваком кораку тежине додатно смањују за део своје вредности, ова техника се назива и *weight decay*.

2. ***L1* регуларизација**
   Уместо квадрата, кажњава се апсолутна вредност тежина

$$L' = L + \lambda\|W\|_1 = L + \lambda \sum_i |w_i|$$

   За разлику од *L2* регуларизације, *L1* регуларизација већи број тежина своди на 0, тако да се добијају ретке мреже.

3. ***Dropout***
   Током тренирања у сваком кораку се насумично искључује део неурона, при чему се сваки неурон искључује са вероватноћом p [40]. На овај начин мрежа не може да се ослони на поједине неуроне, већ мора научити законитости међу свим неуронима. Овако практично тренирамо велики број различитих неуронских мрежа, формирајући ансамбл предиктора. Током инференције се користе сви неурони, а излази се скалирају тако да њихова очекивана вредност буде иста као током тренирања.

4. **Нормализација слојева**
   Batch normalization [41] нормализује улазе слоја тако да имају средњу вредност 0 и

варијансу 1, рачунато над примерима у *mini-batch*-у. Код трансформера и рекурентних мрежа чешће се користи *layer normalization* [42], где се нормализација врши над свим неуронима једног слоја, за сваки пример посебно:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

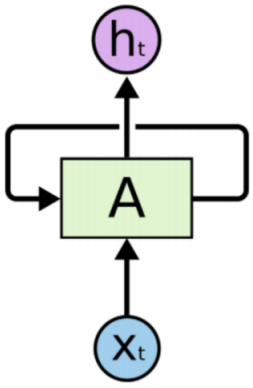
$$\text{LayerNorm}(x) = \gamma\frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta$$

при чему су γ и β параметри који се уче. Нормализација је првенствено уведена како би се стабилизовало и убрзало тренирање, али *batch normalization* има и благ регуларизациони ефекат, услед шума који уноси рачунање средње вредности и варијансе над *mini-batch*-ом. *Layer normalization* ћемо касније срести и код трансформера и код *COGMEN* модела.

5. **Рано заустављање**
   Током тренирања прати се губитак над валидационим скупом и тренирање се зауставља када губитак престане да опада током одређеног броја епоха. Као коначне тежине узимају се оне са најмањим губитком над валидационим скупом.

3.2 Рекурентне неуронске мреже

Код стандардних неуронских мрежа димензије података које мрежа обрађује морају бити познате унапред. Код података који могу бити различитих димензија (нпр. текст различите дужине, звук различитог трајања) ово представља проблем. Као решење овог проблема јавиле су се рекурентне неуронске мреже, које имају повратне гране у скривеним слојевима (слика 12). На овај начин мрежа одржава скривено стање које сажима претходно виђене улазе, чиме се омогућава обрада секвенци произвољне дужине, корак по корак [25].



Слика 12: Ћелија рекурентне неуронске мреже

У наредним одељцима ћемо анализирати основне рекурентне неуронске мреже, као и проблеме који настају при њиховом тренирању, а затим ћемо детаљно обрадити трансформере, који су довели до револуције великих језичких модела (*LLM*) у току последње деценије.

3.2.1 Основна архитектура рекурентних неуронских мрежа

Најосновнија рекурентна неуронска мрежа, као и друге неуронске мреже, на улазу добија сигнал x и генерише излаз y. Разлика у односу на стандардне неуронске мреже је у томе што се излаз y не добија само на основу улаза x и тежина мреже, већ и на основу претходног излаза мреже. Као интуитивно полазиште, излаз мреже у тренутку t може се посматрати као функција тренутног улаза и претходног излаза:

$$y_t = \psi(W_x^T x_t + W_y^T y_{t-1} + b)$$

при чему је ψ активациона функција.

Међутим, у пракси се уводи посебно скривено стање h_t, које моделује интерно стање мреже одвојено од излаза, чиме се омогућава сложеније моделовање на основу улаза и претходног стања [25]:

$$h_t = f(x_t, h_{t-1})$$

На овај начин се предвиђање излаза мреже и памћење претходног стања могу раздвојити:

$$h_t = \psi(W_{xh}^T x_t + W_{hh}^T h_{t-1} + b_h)$$

$$y_t = \psi(W_{hy}^T h_t + b_y)$$

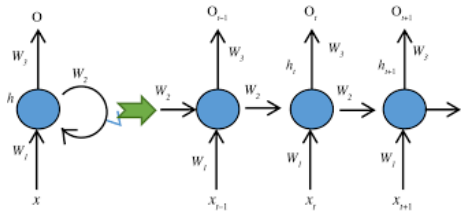
Уколико бисмо покушали да тренирамо овакву рекурентну неуронску мрежу, било би потребно обавити n пролаза кроз мрежу како бисмо добили резултате за целу секвенцу дужине n, а затим извршити пропагацију грешке уназад кроз време (*backpropagation through time*), тј. променити тежине тако да се у скривеном стању мреже кодирају информације потребне за инференцију [25].

У случају $h_t = w \cdot h_{t-1}$, применом ланчаног правила добијамо

$$\frac{\partial h_t}{\partial w} = h_{t-1} + w \cdot \frac{\partial h_{t-1}}{\partial w} = h_{t-1} + w \cdot \left(h_{t-2} + w \cdot \frac{\partial h_{t-2}}{\partial w} \right) = \ldots$$

односно, пошто граф мреже садржи петљу, израз се може развијати бесконачно, јер не постоји базни случај.

Решење овог проблема је да се граф који описује рекурентну неуронску мрежу претвори у ациклични граф, процедуром која се назива *unrolling* (слика 13). Овај процес уклања циклус тако што прави n копија мреже, при чему i-та копија прима улаз x_i, а на излазу даје y_i и h_i, које се затим води на улаз $(i+1)$-е копије. Такође, овим процесом се дефинише базни случај (почетно скривено стање h_0), што омогућава пропагацију грешке уназад.



Слика 13: *Unrolling* рекурентне неуронске мреже

Проблем настаје када мрежу желимо да користимо за дуге секвенце. Размотримо

случај када имамо само 100 копија, што је у реалним случајевима и даље кратка секвенца. Грешку коју добијемо на крају мреже пропагирамо кроз мрежу до прве копије и применом ланчаног правила добијамо израз

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_{100}} \cdot \frac{\partial h_{100}}{\partial h_{99}} \cdot \frac{\partial h_{99}}{\partial h_{98}} \cdot \ldots \cdot \frac{\partial h_2}{\partial h_1}$$

У рекурентним неуронским мрежама се при рачунању скривеног стања користи активациона функција, а као пример ћемо посматрати функцију *tanh*. Извод ове функције има максималну вредност 1, и то само када је улаз једнак 0. Чак и ако би извод у сваком кораку био релативно висок, на пример 0.9, услед великог броја множења долази до **нестајања градијента**, јер је $0.9^{100} \approx 0.0000266$ [43], [44]. Тренирање почетних корака секвенце је тиме практично немогуће.

Супротан проблем настаје када је сваки од чинилаца у горњем производу већи од 1, што се дешава када матрица тежина W_{hh} има велике сопствене вредности [44]. Ако је чинилац и мало већи од 1, рецимо 1.1, за прву копију добијамо $1.1^{100} \approx 13780$. Овај проблем се назива **експлозија градијента**.

Због ова два проблема класичне рекурентне мреже нису нашле примену ван врло кратких секвенци. Као бољи модел, отпоран на нестајање градијента, јавио се *LSTM* [45].

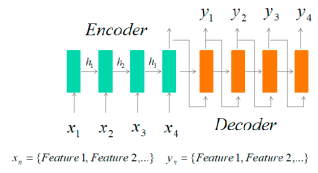
3.2.2 Трансформер архитектура

Seq2Seq модели

Почетком 2010-их година доминантни модели за обраду природних језика били су састављени од *LSTM* ћелија, које су градиле енкодерске и декодерске мреже [46] (слика 14).

- *Encoder*
  Прву мрежу представља енкодер, имплементиран као низ *LSTM* ћелија, који на улазу добија елементе секвенце и производи вектор који треба да обухвати контекст улазне секвенце. Овако добијен вектор се може користити у вишеслојном перцептрону за задатке као што је анализа сентимента, или се може довести на декодер за генеративне задатке.

- *Decoder*
  Вектор који произведе енкодер доводи се на улаз декодера, који затим производи излаз. Енкодер-декодер мреже су биле посебно корисне за превођење језика и генерисање текста, међутим највећи проблем су и даље представљале дуже секвенце, услед ограничења *LSTM* мрежа, као и чињеница да је немогуће задржати целокупан контекст дужег текста у оквиру једног вектора.



Слика 14: *Seq2Seq* модел

Attention механизам

Као решење овог проблема јавио се *attention* механизам [47]. Циљ овог механизма је да омогући моделу да учи зависност сваког излаза мреже од сваког претходног улаза, решавајући на тај начин проблем губљења контекста код дужих секвенци. Интуитивно, овај механизам се може схватити као процес сличан људском разумевању текста, на пример током превођења. Уколико у току читања текста наиђемо на заменицу, покушаћемо да је повежемо са особом која је субјекат радње у претходном тексту, уместо са свим претходно прочитаним речима.

Како овакав „фокус“ имплементирати математички? Трансформери [48] ову функционалност имплементирају помоћу матрица упита (*query*), кључева (*key*) и вредности (*value*), које се добијају пројекцијом матрице X, чији су редови *embedding* вектори елемената улазне секвенце.

- ***Key (K)***
  Матрица K садржи кључеве, односно репрезентације елемената секвенце на основу којих се одређује колико је сваки елемент релевантан за дати упит:

  $$K = XW_K$$

  Матрица W_K се учи током тренирања модела.

- ***Query (Q)***
  Матрица Q представља „питања“ која сваки елемент секвенце поставља, а производом QK^T добија се мера релевантности сваког елемента секвенце у односу на сваки други елемент. Ова „питања“ представљају различите зависности између елемената секвенце, а у области обраде природних језика примери оваквих питања могу бити провера ком глаголу припада субјекат, на кога се односи заменица итд.

  $$Q = XW_Q$$

  Матрица W_Q се учи током тренирања модела.

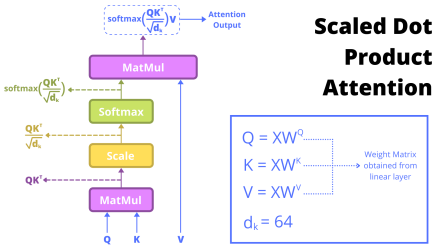
- ***Value (V)***
  Матрица V садржи вредности, односно информације које сваки елемент секвенце преноси осталим елементима када им је релевантан:

  $$V = XW_V$$

  Матрица W_V се такође учи током тренирања.

На основу претходне три матрице рачуна се *attention*, односно на које делове секвенце треба обратити пажњу: матрице Q и K^T се множе, резултат се скалира са $\sqrt{d_k}$, где је d_k димензија вектора кључева, и примењује се *softmax* функција, како би се вредности нормализовале у вероватноће [48] (слика 15). Дељење са $\sqrt{d_k}$ спречава засићење *softmax* функције, до којег долази када су вредности производа QK^T превелике и које би довело до нестајања градијената.

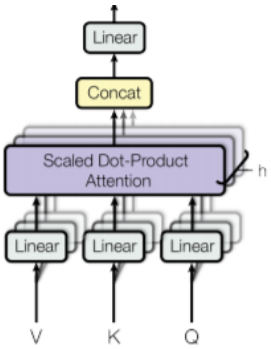
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Слика 15: *Attention* механизам

Као унапређење *attention* механизма јавио се *multi-head attention* (слика 16). Једна *attention* функција омогућава моделу да научи само једну врсту зависности између делова секвенце. Због тога се паралелно користи више *attention* глава (*heads*), које примају исти улаз, али имају сопствене матрице W_Q, W_K и W_V, па свака од њих може да научи другачију зависност. Излази свих глава се затим спајају и пројектују матрицом W_O [48]:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$



Слика 16: *Multi-head attention*

У зависности од тога да ли се *attention* примењује над елементима исте секвенце или између две различите секвенце, овај механизам се дели на *self-attention* и *cross-attention*. Код *cross-attention* механизма упити Q потичу из једне секвенце (нпр. из декодера), а кључеви K и вредности V из друге (нпр. из излаза енкодера).

Како трансформери немају рекурентност, информација о редоследу елемената секвенце уноси се кроз позиционо кодирање, које се додаје на *embedding* векторе пре уласка у мрежу [48].

3.3 Граф неуронске мреже

У наредним одељцима размотрићемо граф неуронске мреже, које се могу користити код задатака са графовским подацима, као што су анализа друштвених мрежа, предвиђање својстава молекула и генерално задаци где се између података могу успоставити релације које се затим могу представити графом.

3.3.1 Опште карактеристике граф неуронских мрежа

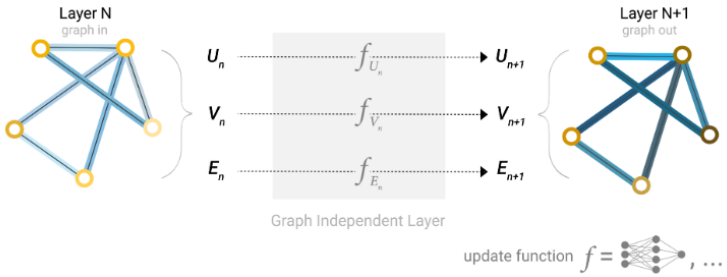
Најосновније граф неуронске мреже на улазу примају граф, а на излазу генеришу трансформисане репрезентације чворова, ивица и глобалних карактеристика графа, до-

бијене применом узастопних трансформација без промене структуре улазног графа [49] (слика 17). Трансформације се врше над три компоненте графа:

- Ивице (E – *Edges*)
  Матрица ивица E чува особине ивица, које могу бити задате скаларом, али се најчешће представљају *embedding* векторима. Пример оваквог *embedding* вектора је код предвиђања својстава молекула, где матрица ивица кодира информације о хемијским везама (тип везе, растојање између атома итд.) за сваку ивицу.

- Чворови (V – *Vertices*)
  Матрица чворова V чува особине чворова, које такође могу бити задате скаларом, али се најчешће, као и код ивица, представљају *embedding* векторима. Један пример је анализа друштвених мрежа, где матрица чворова кодира информације о корисницима (пол, старост, интересовања итд.).

- Глобалне карактеристике графа (U – *Universal*)
  Кодирају карактеристике графа као целине, као што су глобална структура графа, повезаност итд., које се на крају могу користити за предвиђање својстава графа.



Слика 17: Граф неуронска мрежа

Најпростија граф неуронска мрежа дефинише по један вишеслојни перцептрон за сваки од улаза U, V и E и на излазу генерише ажуриране верзије ових матрица. На основу коначних матрица може се вршити предвиђање на нивоу чворова, ивица или целог графа.

Проблем код овакве архитектуре јавља се у задацима где је потребно анализирати једну од три компоненте графа (нпр. чворове), а располажемо подацима о другим компонентама (нпр. ивицама). Решење овог проблема је *pooling*, који се врши у два корака:

1. **прикупљање** *embedding* вектора које желимо да проследимо и њихово спајање у матрицу;
2. **агрегација** прикупљених *embedding* вектора, најчешће сумирањем.

Код примера анализе чворова без информација о њима, могуће је прикупити репрезентације ивица, а затим у сваком чвору сумирати репрезентације његових суседних ивица. У даљем тексту ова операција ће бити означена са

$$\rho_{X \to Y}$$

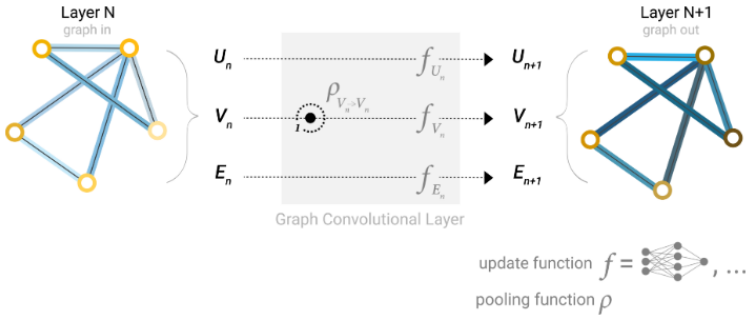
што означава *pooling* података X (нпр. ивица) у Y (нпр. чворове).

Овакав *pooling* је могуће применити у сваком слоју граф неуронске мреже, и то и над истим типом података. Овом техником се омогућава размена информација између

суседних чворова или суседних ивица у графу, чиме се у *embedding* векторима кодира и повезаност графа. Овакав механизам, уз додато учење ажурирања репрезентација, назива се *message passing* [50] и обавља се у три корака (слика 18):

1. за сваки чвор или ивицу прикупљају се репрезентације суседних чворова или ивица;
2. прикупљене репрезентације се агрегирају функцијом агрегације (нпр. сумирањем);
3. агрегиране репрезентације пролазе кроз *update* функцију, најчешће неуронску мрежу.

Понављањем *message passing*-а n пута, сваки чвор добија информације из чворова удаљених до n корака.



Слика 18: *Message passing*

3.3.2 Варијације граф неуронских мрежа

Граф конволуционе и релационе граф конволуционе неуронске мреже

Како се конволуција показала изузетно успешном код задатака из рачунарског вида и обраде звука, природно се поставило питање да ли се може применити и над графовским подацима. Конволуција над графовима се дефинише нешто другачије, преко Лапласијана графа

$$L = D - A$$

где је A матрица суседства, а D дијагонална матрица степена чворова.

Ова операција је изузетно скупа, јер захтева рачунање сопствених вектора Лапласијана, па је уведена апроксимација преко нормализоване агрегације суседа [51]:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

при чему је:

- $H^{(l)}$
  матрица репрезентација чворова у слоју l. Сваки ред ове матрице је *embedding* вектор једног чвора.
- $W^{(l)}$
  матрица тежина слоја l. Ове тежине се уче током тренирања.
- $\tilde{A} = A + I$
  матрица суседства графа са додатим повратним везама, тј. за сваки чвор у графу додаје се ивица која повезује чвор са самим собом.

- $\tilde{D}$
  дијагонална матрица степена чворова у графу са додатим повратним везама, тј. $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$. Ова матрица служи за нормализацију агрегације, јер би чворови са великим бројем суседа након агрегације имали велике вредности.

- σ
  нелинеарна активациона функција, нпр. *ReLU*.

Као што можемо видети, за све ивице у слоју l користи се иста матрица тежина $W^{(l)}$. То значи да су стандардне граф конволуционе неуронске мреже у стању да уче само над графовима који имају једну врсту релација између чворова. Овај проблем решавају релационе граф конволуционе мреже [52], које горњи израз проширују на следећи начин:

$$h_i^{(l+1)} = \sigma\left(W_0^{(l)} h_i^{(l)} + \sum_{r \in R} \sum_{j \in N_i^r} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)}\right)$$

при чему је:

- $h_i^{(l+1)}$
  репрезентација чвора i у слоју $l+1$;
- $W_0^{(l)}$
  матрица тежина за сопствене везе, због чега се у изразу множи са репрезентацијом чвора i у слоју l;
- $r \in R$
  r представља један тип релације између чворова, док је R скуп свих могућих релација;
- $j \in N_i^r$
  j представља чвор који дели ивицу са чвором i, при чему ивица представља релацију r;
- $W_r^{(l)}$
  матрица тежина за релацију r;
- $c_{i,r}$
  нормализациона константа, често постављена на $|N_i^r|$, односно на број суседа са којима чвор i има релацију r.

Граф трансформери

Код стандардних граф неуронских мрежа користи се *message passing* механизам, који смо обрадили у претходном одељку. Може се повући паралела између овог механизма и начина рада стандардних рекурентних неуронских мрежа:

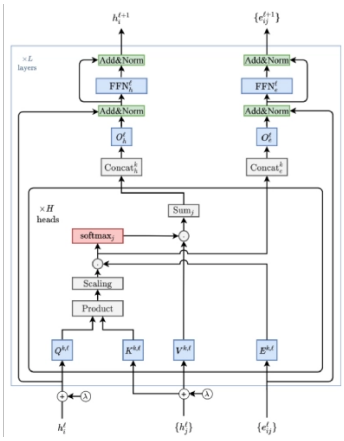
- код рекурентних неуронских мрежа, да бисмо у n-том елементу секвенце добили информацију о првом елементу, морамо проћи кроз првих $n-1$ корака, у сваком од којих се у скривеном стању кодирају претходно обрађени подаци;
- код граф неуронских мрежа, да бисмо у неком чвору или ивици графа добили информацију о чвору или ивици удаљеним n корака, морамо проћи кроз чворове удаљене $1, 2, \ldots, n-1$ корака.

Као и код рекурентних мрежа, овде се јавља неколико проблема:

- **Деградација контекста**
  Уколико желимо да мрежа научи зависности између удаљених делова графа (односно секвенце код *RNN*-а), репрезентације морамо проследити кроз више корака, чиме се губи контекст, односно подаци о удаљеним чворовима или ивицама. Сличан проблем се јавља и када чвор има много суседа, јер се агрегацијом нужно губе подаци о њима – тешко је сажети репрезентације великог броја суседа у један вектор. Овај проблем се у литератури назива *over-squashing* [53].

- ***Oversmoothing***
  Приликом размене репрезентација између чворова врши се њихова агрегација (сумирање) и примена активационе функције. Понављањем размене порука репрезентације чворова постају све сличније, тј. конвергирају ка истој вредности, чиме се поново губи контекст [54].

Слично решење као код рекурентних мрежа може се применити и овде, тј. може се увести *attention* механизам [55], [56]. Код оваквих граф неуронских мрежа као упит Q користи се *embedding* вектор чвора i у слоју l, $h_i^{(l)}$, док се као кључеви K и вредности V користе *embedding* вектори суседних чворова $h_j^{(l)}$ у истом слоју. Уколико располажемо и подацима о ивицама између ових чворова, они се могу искористити пре примене *softmax* функције. На излазу се добијају ажуриране репрезентације чворова и ивица. На овај начин модел учи колико је сваки сусед важан за дати чвор, уместо да све суседе третира једнако, а додавањем више *attention* глава ова архитектура је у стању да учи више различитих зависности (слика 19).



Слика 19: Граф трансформер неуронска мрежа

4. Граф неуронске мреже за препознавање емоција

Претходни модел који смо анализирали је постизао одличне резултате код *embedded* уређаја, међутим могуће је развити комплексније моделе са бољим перформансама. У следећем поглављу размотрићемо модел дат у раду *COGMEN:COntextualized GNN based Multimodal Emotion recognitioN*, који је базиран на граф неуронским мрежама и трансформер архитектури. Овај модел постиже тренутно најбоље резултате у мулдимодалном препознавању емоција.

4.1 Увод

Многи мултимодални модели за препознавање емоција, слично као конволуциони модел из претходног поглавља, врше предикцију емоција на основу видеа и звука само у току једног изговарања. Због тога, оваквим моделима би тешко било да препознају емоције током разговора између две или више особа, где емоције говорника зависе од више фактора као што су контекст разговора, унутрашње стање говорника и онога што су други говорници изговорили.

Због ових ограничења и комплексности људских емоција у току конверзације, настао је модел *COGMEN* у раду *COGMEN:COntextualized GNN based Multimodal Emotion recognitioN*. Овај модел је мултимодалан и за предикцију емоција користи не само видео и аудио већ и текстуални запис разговора. Поред тога, како би се боље моделовао разговор и утицај конверзације на емоције корисника моделоваће се 3 типа информација

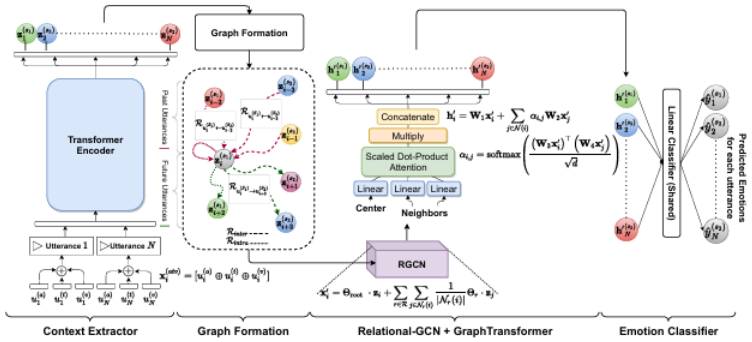
- **Глобални контекст**
  Уколико је контекст, односно тема разговора срећна, у току разговора ће претежно бити позитивних емоција као што су срећа или изненађење. Услед ове чињенице, модел мора бити у стању да на основу конверзације извуче глобални контекст пре предикције.
- **Унутрашње стање говорника**
  Емоције говорника, односно унутрашње стање говорника у неком тренутку конверзације су значајно условљене претходним унутрашњим стањем говорника. За моделовање унутрашњег стања говорника у тренутку t користићемо све информације које је говорник пружио (снимци израза лица, изговорени звук, текст изговора) до тог тренутка t.
- **Утицај других говорника**
  Наравно, велики утицај на емоције говорника има оно што су други говорници изразили. Изрази лица, тон гласа као и саме речи које други говорници изговарају значајно утичу на унутрашње стање посматраног говорника. Како би моделовали унутрашње стање говорника у тренутку t користићемо све информације које су сви остали говорници пружили (снимци израза лица, изговорени звук, текст изговора) до тог тренутка t.

На основу ових 3 информација, аутори предлажу архитектуру која на основу трансформер енкодера, релационе граф неуронске мреже и граф трансформера врши предикцију емоција сваког од говорника у сваком од изговарања. Ова архитектура је у различитим експериментима показала, у тренутку издавања рада, најбоље резултате.

Даље ћемо детаљно обрадити дефиницију релација изговора, архитектуру модела као и хиперпараметре које су аутори користили како би постигли дате резултате.

4.2 Архитектура решења

На слици испод можемо видети целокупну архитектуру *COGMEN* модела.



Слика 20: Архитектура $COGMEN$а

Модел ћемо поделити на 5 дела

- Контекст екстрактор
- Формирање графа
- Релациона граф неуронска мрежа
- Граф трансформер
- Класификатор емоција

4.2.1 Контекст екстрактор

Први део модела је контекст екстрактор, чији је задатак извући контекст односно репрезентацију сваког од изговора који чине видео, аудио као и текст. За i-ти податак

$$u_i; i = 1, \dots n$$

можемо издвојити 3 компоненте

- аудио
$$u_i^{(a)} \in \mathbb{R}^{d_a}$$
- текст
$$u_i^{(t)} \in \mathbb{R}^{d_t}$$
- видео
$$u_i^{(v)} \in \mathbb{R}^{d_v}$$

На основу ових компоненти можемо дефинисати улазни *feature*

$$x_i^{atv} = [u_i^{(a)} \oplus u_i^{(t)} \oplus u_i^{(v)}] \in \mathbb{R}^d,\ d = d_a + d_t + d_v$$

Сви улази формирају улазну матрицу

$$X = x^{atv} = [x_1^{atv}, x_2^{atv}, \dots, x_n^{atv}]^T$$

Овако формирана матрица се доводи на улаз трансформер енкодера. Као што смо већ дефинисали код поглавља о трансформер архитектури, потребне су нам три матрице

Q, K и V за *attention* механизам. Помоћу *attention* механизма ћемо омогућити моделу да учи зависности између појединачних изговора и на основу тога енкодирати глобални контекст разговора. Како би модел учио више различитих релација, додаје се више *attention head*а, при чему индекс *head*а је дат са h.

$$\begin{aligned} Q^{(h)} &= XW_{h,q} \\ K^{(h)} &= XW_{h,k} \\ V^{(h)} &= XW_{h,v} \\ W_{h,q},\ W_{h,k},\ W_{h,v} &\in \mathbb{R}^{d\times k} \end{aligned}$$

Тада су *attention* тежине дате са

$$\alpha^{(h)} = \sigma_j\left(\frac{Q^{(h)}(K^{(h)})^T}{\sqrt{k}}\right),\ \alpha^{(h)} \in \mathbb{R}^{n\times n}$$

На основу $\alpha^{(h)}$ можемо одредити излаз сваког *attention head*а, као и њихову тежинску суму.

$$\begin{aligned} head^{(h)} &= \alpha^{(h)}(V^{(h)}) \in \mathbb{R}^{n\times k} \\ U' &= [head^{(1)} \oplus head^{(2)} \oplus ... head^{(H)}]W^0 \end{aligned}$$

Матрица U' се рачуна као тежинска сума свих излаза *attention* механизма, како би модел могао да научи која од *attention head*а ће најбоље утицати на резултате. На крају трансформер енкодера ћемо матрици U' додати резидуалну конекцију и применити *LayerNorm*.

$$\begin{aligned} U &= LayerNorm(X + U'; \gamma_1, \beta_1) \\ Z' &= ReLU(UW_1)W_2 \\ Z &= LayerNorm(U + Z'; \gamma_2, \beta_2) \\ \gamma_1, \beta_1, \gamma_2, \beta_2 &\in \mathbb{R}^{\mathbb{d}} \\ W_1 \in \mathbb{R}^{\mathbb{d}\times\mathbb{m}},\ W_2 &\in \mathbb{R}^{\mathbb{m}\times\mathbb{d}} \end{aligned}$$

На крају ове компоненте добијамо излаз који представља енкодирани *feature* за сваки од улазних изговора.

$$[z_1, z_2, ..., z_n]^T = Z \in \mathbb{R}^{\mathbb{n}\times\mathbb{d}}$$

4.2.2 Формирање графа

Сваки од излаза z_i контекст екстрактора ће представљати један чвор у графу изговора. Ивице између чворова ће представљати једну од две могуће релације тј. интра и интер релације.

Формално можемо дефинисати разговор M говорника на следећи начин

$$\mathcal{D} = \{\mathcal{U}^{S_1}, \mathcal{U}^{S_2}, ... \mathcal{U}^{S_M}\}$$

при чему

$$\mathcal{U}^{S_1} = \{u_1^{(S_1)}, u_2^{(S_1)}, ... u_n^{(S_1)}\}$$

представља скуп изговора говорника 1.

Тада можемо дефинисати

- **Интра** релације
  односно релације између тренутног изговора и свих осталих изговора посматраног говорника
$$\mathcal{R}_{intra} \in \{\mathcal{U}^{\mathcal{S}_i} \rightarrow \mathcal{U}^{S_i}\}$$
- **Интер** релације
  односно релације између тренутног изговора и свих осталих изговора других говорника.
$$\mathcal{R}_{inter} \in \{\mathcal{U}^{\mathcal{S}_i} \rightarrow \mathcal{U}^{S_j}\}_{i \neq j}$$

Посматрати све прошле и будуће изговоре је практично неизводљиво, нарочито код дугих конверзација тако да се уводе величине прозора за прошле ($\mathcal{P}$) и будуће ($\mathcal{F}$) изговоре, који представлљају хиперпараметре.

4.2.3 Релациона граф неуронска мрежа

Након креирања графа у претходном кораку, следећа компонента је релациона граф неуронска мрежа. Циљ ове компоненте је да, на основу упостављених изговора (чворови у графу) и релација између њих (ивице), акумулира репрезентације чворова преко *message passing* механизма у зависности од релација између њих.

Ажурирање репрезентација се врши на следећи начин

$$x'_i = \theta_{root} \cdot z_i + \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{\|\mathcal{N}_r(i)\|} \theta_r \cdot z_j$$

где је

- $\mathcal{N}_r(i)$ скуп суседа чвора i са којима има релацију r
- $\theta_{root}, \theta r$ су параметри релационе граф неуронске мреже који се уче
- $\|\mathcal{N}_r(i)\|$ је нормализациона константа, једнака броју суседа
- $z_i,\ z_j$ су тренутни изговор и изговор суседа j, респективно

4.2.4 Граф трансформер

Акумулацијом репрезентација добијамо финалне вредности чворова

$$H = x'_1, x'_2, ... x'_n$$

на основу којих можемо добити репрезентације за даље предвиђање у коначном слоју помоћу граф трансформера. Излази граф трансформера ће бити

$$h'_i = W_1 x'_i + \sum_{j \in \mathcal{N}_{(i)}} \alpha_{i,j} W_2 x'_j$$

где су $\alpha_{i,j}$ *attention* коефицијенти између i-тог и j-тог чвора и рачунају се

$$\alpha_{i,j} = softmax\left(\frac{(W_3 x'_i)^T (W_4 x'_j)}{\sqrt{d}}\right)$$

4.2.5 Класификатор емоција

На крају модела имамо линеарни слој за предикцију излаза на основу излаза граф трансформера. Излаз се одређује на следећи начин

$$h_i = ReLU(W_1 h'_i + b_1)$$
$$\mathcal{P}_i = softmax(W_2 h_i + b_2)$$
$$\hat{y_i} = argmax(\mathcal{P}_i)$$

4.2.6 Хиперпараметри модела

Аутори су предложили хиперпараметре модела за случај тренирања модела на *IEMOCAP* и *MOSEI* скуповима података.

- За *IEMOCAP* скуп података, аутори предлажу следеће параметре
  - *Dropout*: 0.1
  - Број *attention* глава: 7
  - *SeqContext* (број слојева у контекст енкодеру): 4
  - Иницијална стопа учења (*learning rate*): 1^{-4}
- За *MOSEI* скуп података података, имамо 3 случаја
  - Тренинг са само текстом
    - *Dropout*: 0.399
    - Број *attention* глава: 3
    - *SeqContext* (број слојева у контекст енкодеру): 5
    - Иницијална стопа учења (*learning rate*): 3.3^{-3}
  - Тренинг са аудиом и текстом
    - *Dropout*: 0.103
    - Број *attention* глава: 1
    - *SeqContext* (број слојева у контекст енкодеру): 2
    - Иницијална стопа учења (*learning rate*): 6.9^{-3}
  - Тренинг са аудиом, видеом и текстом
    - *Dropout*: 0.337
    - Број *attention* глава: 2
    - *SeqContext* (број слојева у контекст енкодеру): 12
    - Иницијална стопа учења (*learning rate*): 1.1^{-3}

Што се тиче прозора $\mathcal{F}$ і $\mathcal{P}$, аутори су експериментално установили да најбоље резултате дају прозори од 4 и 10 изговора. Мањи прозори дају боље резултате код разговора где се тема често мења, док код дужих разговора боје резултате дају већи прозори.

Modalities	Window Past	Window future	F1 Score (%)
atv	1	1	81.72
atv	2	2	83.21
atv	4	4	**84.08**
atv	5	5	83.19
atv	6	6	82.49
atv	7	7	82.28
atv	9	9	82.77
atv	10	10	**84.50**
atv	11	11	83.93
atv	15	15	83.78

Слика 21: Перформансе $COGMEN$а у зависности од величине прозора

4.3 Експериментални резултати

Тестирање перформанси модела је вршено на два скупа података

4.3.1 *IEMOCAP*

Мултимодални скуп података за препознавање емоција где је сваки изговор у скупу података класификован са једном од 6 емоција - **бес, узбуђење, туга, срећа, фрустрација и неутрално стање**. У литератури се овај скуп података често користи за тестирање на два начина, један где се узима подскуп од 4 емоција (бес, туга, срећа, неуторално стање) као и пун скуп са 6 емоција.

За овај скуп података, аудио *feature*и имају 100 компоненти и екстрактовани су помоћу *OpenSmile*, *open-source* скуп алата за обраду звука. Видео *feature*и су величине 512 и узети су из рада *OpenFace 2.0: Facial Behavior Analysis Toolkit* док су текстуални *feature*и величине 768 екстрактовани помоћу *sBERT* модела.

Тестирање са 6 емоција

Приликом тестирања са 6 емоција и упоређивањем са другим моделима у које спадају модели као што су *memnet*, *TFN*, *MFN*, *DialogueRNN* и други познати модели за предвиђање емоција, *COGMEN* постиже најбољу просечну прецизност и просечан *F1* резултат, око 4% боље у односу на у том тренутку најбоље моделе. Анализом појединачних емоција, *COGMEN* у већини случајева постиже најбоље перформансе, изузев код беса и фрустрације.

Models	IEMOCAP: Emotion Categories							
	Happy	Sad	Neutral	Angry	Excited	Frustrated	Avg.	
	F1 (%)	F1 (%)	F1 (%)	F1 (%)	F1 (%)	F1 (%)	Acc. (%)	F1 (%)
bc-LSTM	35.6	69.2	53.5	66.3	61.1	62.4	59.8	59.0
memnet	33.0	69.3	55.0	66.1	62.3	63.0	59.9	59.5
TFN	33.7	68.6	55.1	64.2	62.4	61.2	58.8	58.5
MFN	34.1	70.5	52.1	66.8	62.1	62.5	60.1	59.9
CMN	32.6	72.9	56.2	64.6	67.9	63.1	61.9	61.4
ICON	32.8	74.4	60.6	68.2	68.4	**66.2**	64.0	63.5
DialogueRNN	32.8	78.0	59.1	63.3	73.6	59.4	63.3	62.8
CAN	31.8	71.9	60.4	66.7	68.5	66.1	63.2	62.4
Af-CAN	37.0	72.1	60.7	**67.3**	66.5	66.1	64.6	63.7
COGMEN	**51.9**	**81.7**	**68.6**	66.0	**75.3**	58.2	**68.2**	**67.6**

Слика 22: Перформансе $COGMEN$ модела на $IEMOCAP$ скупу са 6 емоција

Тестирање са 4 емоција

У случају тестирања над подскупом са 4 емоција, аутори су поредили перформансе *COGMEN*а са два друга модела - *bc-LSTM* и *CHFusion*. Овде *COGMEN* постиже још већу

разлику у односу на остале моделе, где доноси побољшање од 7.7%.

Model	F1-score (%)
bc-LSTM	75.13
CHFusion	76.80
COGMEN	**84.50**

Слика 23: Перформансе $COGMEN$ модела на $IEMOCAP$ скупу са 4 емоција

4.3.2 *MOSEI*

MOSEI је мултимодални скуп података за предвиђање емоција где сваки изговор има две лабеле

- Емоција - срећа, туга, гађење, страх, изненађење и бес
- Сентимент - вредност између -3 (врло негативан) до $+3$ (веома позитиван)

Аудио *feature*и су добијени из рада *A Transformer-based joint-encoding for Emotion Recognition and Sentiment Analysis* и представљени су векторима величине 80. Видео *feature*и су узети из рада *Multimodal language analysis in the wild: Cmu-mosei dataset and interpretable dynamic fusion graph*, величине 35 док су текстуални *feature*и величине 768 екстрактовани помоћу *sBERT* модела.

Код овог скупа података, аутори користе два режима тестирања

- **Бинарна класификација**, где се по један модел тренира за сваку од емоција
- **Multi-label** класификација, где свака реченица има више лабела и један модел превиђа више класа.

Овакво тестирање се врши јер *Multilogue-Net* модел је тестиран са бинарном класификацијом док је *TBJE* тестиран са *multi-label* скупом.

Код случаја са класификацијом сентимента са 2 класе *COGMEN* постиже најбоље резултате од 85% када је трениран над аудиом и текстом. За класификацију сентимента са 7 класа постиже сличне резултате као и други модели.

		Sentiment Class Accuracy(%)		Emotion Class (weighted) F1-score (%)						Multi-label Emotion Class (weighted) F1-score (%)					
Model		2 Class	7 Class	Happiness	Sadness	Angry	Fear	Disgust	Surprise	Happiness	Sadness	Angry	Fear	Disgust	Surprise
Multilogue-Net	T + A + V	82.88	**44.83**	67.84	65.34	67.03	87.79	74.91	**86.05**	70.6	70.7	74.4	86.0	83.4	87.8
TBJE	T	81.9	44.2	-	-	-	-	-	-	63.4	65.8	75.3	84.0	84.5	81.4
	A + T	82.4	43.91	65.91	70.78	70.86	87.79	82.57	86.04	65.5	67.9	76.0	**87.2**	84.5	86.1
	T + A + V	81.5	44.4	-	-	-	-	-	-	64.0	67.9	74.7	84.0	83.6	86.1
COGMEN	T	84.42	43.50	69.28	70.49	73.04	87.80	83.69	85.83	69.92	72.16	77.34	86.39	**86.00**	88.27
	A + T	**85.00**	44.31	68.39	**73.28**	74.98	88.08	**83.90**	85.35	69.62	72.67	76.93	86.39	85.35	88.21
	T + A + V	84.34	43.90	**70.42**	72.31	**76.20**	**88.17**	83.69	85.28	**72.74**	**73.90**	**78.04**	86.71	85.48	**88.37**

Слика 24: Перформансе $COGMEN$ модела на *MOSEI* скупу

Можемо да приметимо да други модели имају горе перформансе када се укључи и видео током тренинга, док је *COGMEN* у стању да извуче релевантне информације и из видеа.

4.4 Оптимизација *COGMEN* модела

Као што смо могли видети у претходним поглављима, *COGMEN* постиже одличне резулате код мултимодалног препознавања емоција што долази уз велику компутациону цену. Услед захтевности овог модела, није могуће применити користити га на ограниченим уређајима као што су мобилни и *embedded* уређаји. Једно потенцијално решење је квантизација, што је и главна идеја рада *Multimodal Emotion Recognition Using Compressed Graph Neural Networks*.

Циљ аутора је био извршити квантизацију модела након тренинга како би се смањила меморијска захтевност модела а тиме и брзина учитавања модела у меморију и инференција. Аутори предлажу два типа квантизације

- **Бинаризација** где се свака тежина модела мења са 1 или -1
- ***FP8* квантизација** где се *float32* вредности мапирају на *float8*

Приликом тестирања перформансе испробана су две приступа примене ових квантизације

- У првом приступу се квантизација примењује само на слојеве са преко милион параметара, којих има 8 код *COGMEN*а
- Други приступ подразумева примену квантизације над целом мрежом.

Перформансе модела пре квантизације су дати испод

	Precision	Recall	F1-score
Macro-averaged value	82.39%	81.24%	81.66%
Weighted average	82.22%	81.97%	81.96%
Average accuracy	81.97%		

Слика 25: Перформансе $COGMEN$ модела пре квантизације

4.4.1 Бинаризација

Бинаризација представља екстреман облик квантизације, где се изворна вредност, најчешће *float32* или *float64*, мења бинарном вредношћу односно може имати саму једно од 2 вредности. Оваква квантизација доноси огромна смањења у меморијској захтевности (32 односно 64 пута мање меморије) а тиме и огромна убрзања. Поред тога, модели који се бинаризују се могу имплементирати логичким колима на *FPGA* хардверу, тиме доносећи још већа убрзања. Проблем код бинаризације је то што мапирањем великог опсега могућих вредности тежина на само две вредности нужно доводи до пада у перформансама.

Аутори користе једноставну детерминистичку функцију за бинаризацију

$$x_b = \text{sign}(x) = \begin{cases} +1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$

Перформансе након бинаризације знатно опадају, при чему бинаризација над слојевима са преко милион параметара постиже у појединим мерењима и до 10% побољшања.

Layers	Over million parameters			All parameters		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Macro-averaged value	49.42%	36.85%	35.25%	38.23%	32.32%	29.03%
Weighted average	48.13%	44.75%	35.25%	39.94%	41.57%	34.64%
Average accuracy	44.75%			41.57%		

Слика 26: Перформансе $COGMEN$ модела након бинаризације

Потенцијални разлог лоших перформанси бинаризованог модела је то што расподела у хистограму тежина модела не прати Лапласову расподелу око нуле, при чему би у том случају бинаризација била погоднија метода квантизације.

4.4.2 *FP8* квантизација

Формат *float8* података који су аутори користили се састоји од

- *Sign* бит-а - 1 бит који означава знак
- Експонент од 5 бита
- Мантиса од 2 бита

Конверзија вредности x_1 се затим врши на следећи начин 1. Одређујемо знак, експонент и мантису

$$e = 5,\ m = 2$$

$$s = \begin{cases} 0 & x_1 \geq 0 \\ 1 & x < 0 \end{cases}$$

$$E_b = \lfloor \log_2(|x_1|) \rfloor$$
$$E_b > 2^{e-1} - 1 \rightarrow E_b = 2^{e-1} - 1$$
$$E_b < -2^{e-1} \rightarrow E_b = -2^{e-1}$$

$$M = \text{round}(2^m(\frac{|x_1|}{2_b^E} - 1))$$
$$M > 2^m - 1 \rightarrow M = 2^m - 1$$
$$M < 0 \rightarrow M = 0$$

2. Одредимо квантизовану вредност

$$x = (-1)^s 2^{E_b} \left(1 + \frac{M}{2^m}\right)$$

Након квантизације, модел задржава практично идентичне перформансе. У овом случају, квантизација свих параметара доноси боље резултате него квантизација само слојева са преко милион параметара.

Layers	Over million parameters			All parameters		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Macro-averaged value	80.90%	81.93%	81.39%	80.99%	82.58%	81.71%
Weighted average	81.86%	81.76%	81.78%	82.15%	81.97%	82.00%
Average accuracy	81.76%			81.97%		

Слика 27: Перформансе $COGMEN$ модела након квантизације на $float8$

Како квантизација доводи до 4 пута мање потребне меморије, одлично је решење за смањење захтевности без пада у перформансама.

Precision	Model size (MB)
Full-precision	103.1
FP8	25.8
Binarization	3.22

Слика 28: Меморијска захтевност $COGMEN$ модела у зависности од квантизације

5. JEPA

У наредном поглављу обрадићемо JEPA (Joint Embedding Predictive Architecture) архитектуру. Циљ ове архитектуре је научити боље репрезентације, односно embedding-е, тако што модел учи да предвиди скривене делове репрезантација или будуће репрезентације на основу тренутних. Принципе рада оваквих модела ћемо касније искористити за потенцијално унапређење COGMEN модела.

5.1 Принцип рада и архитектура JEPA модела

5.2 Варијанте JEPA

5.3 Тренирање JEPA модела

6. Предлог рада

7. Литература

TODO – SREDI LITERATURU

1. S. Poria, E. Cambria, R. Bajpai, and A. Hussain, "A review of affective computing: From unimodal analysis to multimodal fusion," *Information Fusion*, vol. 37, pp. 98–125, 2017.
2. Y. Wang, W. Song, W. Tao, A. Liotta, D. Yang, X. Li, S. Gao, Y. Sun, W. Ge, W. Zhang, and W. Zhang, "A systematic review on affective computing: Emotion models, databases, and recent advances," *Information Fusion*, vol. 83–84, pp. 19–52, 2022.
3. P. Ekman, "An argument for basic emotions," *Cognition and Emotion*, vol. 6, no. 3–4, pp. 169–200, 1992.
4. R. Plutchik, "The nature of emotions," *American Scientist*, vol. 89, no. 4, pp. 344–350, 2001.
5. J. A. Russell, "A circumplex model of affect," *Journal of Personality and Social Psychology*, vol. 39, no. 6, pp. 1161–1178, 1980.
6. A. Mehrabian, "Pleasure-arousal-dominance: A general framework for describing and measuring individual differences in temperament," *Current Psychology*, vol. 14, no. 4, pp. 261–292, 1996.
7. G. A. Miller, "WordNet: A lexical database for English," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.
8. E. Cambria, S. Poria, R. Bajpai, and B. Schuller, "SenticNet 4: A semantic resource for sentiment analysis based on conceptual primitives," in *Proc. COLING 2016*, 2016, pp. 2666–2677.
9. R. Speer, J. Chin, and C. Havasi, "ConceptNet 5.5: An open multilingual graph of general knowledge," in *Proc. AAAI Conference on Artificial Intelligence*, 2017, pp. 4444–4451.
10. A. Balahur, J. M. Hermida, and A. Montoyo, "Building and exploiting EmotiNet, a knowledge base for emotion detection based on the appraisal theory model," *IEEE Transactions on Affective Computing*, vol. 3, no. 1, pp. 88–101, 2012.
11. W. Medhat, A. Hassan, and H. Korashy, "Sentiment analysis algorithms and applications: A survey," *Ain Shams Engineering Journal*, vol. 5, no. 4, pp. 1093–1113, 2014.
12. S. Poria, N. Majumder, R. Mihalcea, and E. Hovy, "Emotion recognition in conversation: Research challenges, datasets, and recent advances," *IEEE Access*, vol. 7, pp. 100943–100953, 2019.
13. C. Busso, M. Bulut, C.-C. Lee, A. Kazemzadeh, E. Mower, S. Kim, J. N. Chang, S. Lee, and S. S. Narayanan, "IEMOCAP: Interactive emotional dyadic motion capture database," *Language Resources and Evaluation*, vol. 42, no. 4, pp. 335–359, 2008.
14. A. Zadeh, P. P. Liang, S. Poria, E. Cambria, and L.-P. Morency, "Multimodal language analysis in the wild: CMU-MOSEI dataset and interpretable dynamic fusion graph," in *Proc. 56th Annual Meeting of the ACL*, 2018, pp. 2236–2246.
15. S. Poria, D. Hazarika, N. Majumder, G. Naik, E. Cambria, and R. Mihalcea, "MELD: A multimodal multi-party dataset for emotion recognition in conversations," in *Proc. 57th Annual Meeting of the ACL*, 2019, pp. 527–536.
16. M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classification: An overview," arXiv:2008.05756, 2020.
17. N. Cummins, S. Scherer, J. Krajewski, S. Schnieder, J. Epps, and T. F. Quatieri, "A review of depression and suicide risk assessment using speech analysis," *Speech Communication*, vol. 71, pp. 10–49, 2015.
18. S. Zepf, J. Hernandez, A. Schmitt, W. Minker, and R. W. Picard, "Driver emotion recog-

nition for intelligent vehicles: A survey," *ACM Computing Surveys*, vol. 53, no. 3, art. 64, 2020.
19. E. Yadegaridehkordi, N. F. B. M. Noor, M. N. B. Ayub, H. B. Affal, and N. B. Hussin, "Affective computing in education: A systematic review and future research," *Computers & Education*, vol. 142, art. 103649, 2019.
20. D. McDuff, R. El Kaliouby, J. F. Cohn, and R. W. Picard, "Predicting ad liking and purchase intent: Large-scale analysis of facial responses to ads," *IEEE Transactions on Affective Computing*, vol. 6, no. 3, pp. 223–235, 2015.
21. "Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," *Official Journal of the European Union*, 2024, Art. 5(1)(f).
22. Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
23. W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, 1943.
24. F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
25. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
26. K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.
27. V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.
28. A. L. Maas, A. Y. Hannun, and A. Y. Ng, "Rectifier nonlinearities improve neural network acoustic models," in *Proc. ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.
29. D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)," in *Proc. International Conference on Learning Representations (ICLR)*, 2016.
30. D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv:1606.08415, 2016.
31. F. van Veen and S. Leijnen, "The neural network zoo," The Asimov Institute, 2019. [Online]. Available: https://www.asimovinstitute.org/neural-network-zoo/
32. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back--propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
33. S. Ruder, "An overview of gradient descent optimization algorithms," arXiv:1609.04747, 2016.
34. H. Robbins and S. Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.
35. B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
36. Y. Nesterov, "A method for solving the convex programming problem with convergence rate $O(1/k^2)$," *Soviet Mathematics Doklady*, vol. 27, pp. 372–376, 1983.
37. D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
38. X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.

39. K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034.
40. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
41. S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
42. J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016.
43. Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
44. R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proc. 30th International Conference on Machine Learning (ICML)*, 2013, pp. 1310–1318.
45. S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
46. I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in Neural Information Processing Systems 27*, 2014, pp. 3104–3112.
47. D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
48. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.
49. B. Sanchez-Lengeling, E. Reif, A. Pearce, and A. B. Wiltschko, “A gentle introduction to graph neural networks,” *Distill*, 2021.
50. J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proc. 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1263–1272.
51. T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. International Conference on Learning Representations (ICLR)*, 2017.
52. M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling relational data with graph convolutional networks,” in *The Semantic Web – ESWC 2018*, LNCS, vol. 10843, 2018, pp. 593–607.
53. U. Alon and E. Yahav, “On the bottleneck of graph neural networks and its practical implications,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021.
54. Q. Li, Z. Han, and X.-M. Wu, “Deeper insights into graph convolutional networks for semi-supervised learning,” in *Proc. AAAI Conference on Artificial Intelligence*, 2018, pp. 3538–3545.
55. P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. International Conference on Learning Representations (ICLR)*, 2018.
56. Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, “Masked label prediction: Unified message passing model for semi-supervised classification,” in *Proc. 30th International Joint Conference on Artificial Intelligence (IJCAI)*, 2021, pp. 1548–1554.
57. https://medium.com/@RobuRishabh/recurrent-neural-network-rnn-8412b9abd755

58. https://medium.com/@fredroj/recurrent-neural-network-rnn-cell-internal-mechanism--ff5eca732adb
59. https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/
60. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
61. https://poloclub.github.io/transformer-explainer/
62. https://medium.com/data-science/introduction-to-attention-mechanism-in-deep-learning--eli5-way-d6c2d799ef18
63. https://medium.com/@paulpierre/eli5-transformers-gpt-and-the-power-of-the-attention--mechanic-6504693f2c3f
64. https://www.d2l.ai/chapter_attention-mechanisms-and-transformers/queries-keys-values.html
65. https://apxml.com/courses/introduction-to-transformer-models/chapter-2-self-attention--multi-head-attention/query-key-value-vectors
66. https://kumo.ai/research/introduction-to-graph-transformers/
67. https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization
68. https://www.dgl.ai/dgl_docs/tutorials/models/1_gnn/4_rgcn.html